

COS30018 – Intelligent Systems

Option C – Machine Reading Comprehension (MRC) based on Generative AI

Project Summary Report

Students:

- Nguyen Chanh Hoai Nam (104195191)
- Le Quang Thien (104226017)
- Dinh Xuan Sinh (104180692)
- Le Minh Long (104180139)
- Nguyen Thi Thanh Minh (104169617)

Tutor: Dr. Le Van Minh

Project Summary Report.....	1
I. Introduction	3
II. Overall System Architecture.....	3
A. Presentation Layer	3
B. Application Layer.....	4
C. System Workflow	4
III. Detailed installation/user guides	4
A. Prerequisites	4
B. Installation.....	4
C. Running the program	6
D. Using the Model.....	6
IV. Details of the implementations members of the team have done	6
A. Process Dataset	6
B. Implement Model	8
C. Fine Tune	10
D. Evaluation.....	11
E. UI Design	12
F. Testing	12
V. Detailed Experimentations.....	13
A. Phase 1: Fine-Tuning Different Models with Consistent Hyperparameters.....	13
B. Phase 2: Hyperparameter Tuning for the Best-Performing Model	14
VI. Experimentations Result Analysis and Discussion	14
A. Result Analysis.....	15
B. Discussion.....	17
VII. Implementation Critical Analysis.....	17
A. Strengths and Successes	17
B. Weaknesses and Limitations	19
C. Comparison with Existing Models.....	19
D. Qualitative Test Cases	20
VIII. Conclusion.....	23
IX. References.....	24

I. Introduction

Machine Reading Comprehension (MRC) has become a pivotal area of research in Natural Language Processing (NLP), aiming to develop systems that can understand and answer questions based on a given context. The significance of MRC lies in its wide range of applications, from automated customer service to advanced research assistance. With the advent of large-scale pre-trained language models like BERT (Bidirectional Encoder Representations from Transformers), there has been substantial progress in enhancing the accuracy and efficiency of MRC systems.

This project, undertaken as part of the COS30018 course, focuses on building a robust MRC system using state-of-the-art NLP techniques and tools. Our primary goal is to fine-tune pre-trained language models, specifically BERT and DistilBERT, to accurately answer questions from two datasets: MashQA and COVID-QA. These datasets present diverse challenges, including variations in context length, complexity of questions, and the specificity of the required answers.

To achieve our objectives, we have implemented a comprehensive pipeline that includes data preprocessing, model fine-tuning, evaluation, and a user-friendly graphical interface for real-time question answering. The project leverages the Hugging Face Transformers library for model implementation and fine-tuning, and Streamlit for developing an interactive web application.

The following report provides a detailed overview of our system's architecture, the methodologies employed, experimental setups, and the results obtained. Through rigorous experimentation and analysis, we aim to highlight the strengths and limitations of our approach, offering insights into potential improvements and future directions for MRC systems. This project not only demonstrates the practical application of advanced NLP techniques but also contributes to the growing body of research in machine reading comprehension.

II. Overall System Architecture

Our MRC system employs a two-tier architecture, comprising a presentation layer for user interaction and an application layer handling the core MRC functionality.

A. Presentation Layer

- **Framework:** Streamlit
- **Functionality:**

- Provides a user-friendly interface for inputting context and questions.
- Displays the generated output from the MRC model.

B. Application Layer

- **Core Library:** Hugging Face Transformers
- **Functionality:**
 - Houses the pre-trained BERT model for MRC tasks.
 - Utilizes the AutoTokenizer for text preprocessing.
 - Processes user input (context and question) and generates the corresponding answer.

C. System Workflow

1. **User Interaction:** The user inputs the context and question via the Streamlit interface.
2. **Data Preprocessing:** The input text is tokenized using the AutoTokenizer from Hugging Face Transformers.
3. **Model Inference:** The pre-trained BERT model processes the tokenized input and generates an answer.
4. **Output Display:** The generated answer is presented to the user through the Streamlit interface.

III. Detailed installation/user guides

This guide will help you set up and use the fine-tuned LLM model google-bert/bert-base-cased with MASHQA and COVID datasets. Follow the steps below to install the necessary packages, run the program, and understand its usage.

A. Prerequisites

- Ensure you have Python installed on your system.

B. Installation

1. Step 1: Clone the repository

- Open your terminal and command prompt as administrator
- Navigate to the directory where you want to clone the repository
- Run the following command to clone the repository:
- `git clone https://github.com/hnam26/COS30018.git`

2. Step 2: Create a Virtual Environment

- Navigate to the cloned directory.
- Run the following command to create a virtual environment:

```
python -m venv venv
```

3. Step 3: Activate the Virtual Environment

- On Windows: `.\venv\Scripts\activate`
- On macOS and Linux: `source venv/bin/activate`

4. Step 4: Install Required Packages

- Run the following command to install the necessary packages:

```
pip install -r requirements.txt
```

5. Step 5: Environment variables configuration

- Create a `.env` file in the root directory of the project. This file should contain your Huggingface and Weights & Biases API keys.
- Use the provided `.env.example` file as a template for what your `.env` file should look like.

Instructions on how to get Huggingface and Weights & Biases API keys:

- [Huggingface API key](#)
- [Weights & Biases API key](#)

6. Step 6: Fine-tune the models

- `--train`: List of paths to the train data files. You can provide multiple files by separating the paths with spaces.
- `--val`: List of paths to the validation data files. You can provide multiple files by separating the paths with spaces.
- `--test`: List of paths to the test data files. You can provide multiple test data files by separating the paths with spaces.
- `--model_id`: ID of the model you want to fine-tune. You can find other model IDs on the Huggingface Model Hub.
- `--output_dir`: Path to the directory where you want to save the trained model and its outputs.

Example:

```
python main.py --train train_covid.json train_mash.json --val  
val_mash.json val_covid.json --test test_mash.json test_covid.json --  
model_id bert-base-uncased --output_dir output
```

C. Running the program

We utilized [Streamlit](#) to build a GUI of the model. To run the program, follow these steps:

- Ensure your virtual environment is activated.
- Run the following command to start the program:
- `streamlit run gui/app.py`

D. Using the Model

To ensure the model's accuracy, please use the datasets that were used for its fine-tuning.

- Context Input: Enter the context related to your question in the provided input field.
- Question Input: Type your question in the question input field.

IV. Details of the implementations members of the team have done

This section provides a comprehensive overview of the tasks undertaken by the team to develop the Machine Reading Comprehension (MRC) system. The tasks are categorized into six main areas: dataset processing, model implementation, fine-tuning, evaluation, UI design, and testing. Each section includes the task's purpose, and the methods used.

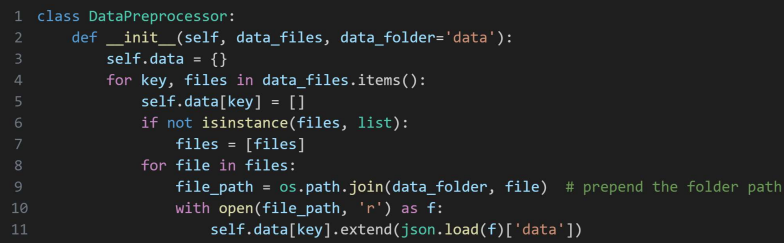
A. Process Dataset

To begin our project, we needed to collect, preprocess, and prepare the datasets for model training. This initial step was crucial to ensure the datasets were in the correct format and ready for fine-tuning, as proper preprocessing enhances the accuracy and efficiency of the training process. This process involved three main steps: loading data, preprocessing data, and preparing the dataset.

1. Loading Data:

To accomplish this, we defined a `DataPreprocessor` class that loads JSON files from a specified folder. This class manages multiple files and supports both training and validation data. The raw data is loaded into memory, creating a dictionary for each dataset split (train, validation, test).

Option C – Machine Reading Comprehension (MRC) based on Generative AI



```

1 class DataPreprocessor:
2     def __init__(self, data_files, data_folders='data'):
3         self.data = {}
4         for key, files in data_files.items():
5             self.data[key] = []
6             if not isinstance(files, list):
7                 files = [files]
8             for file in files:
9                 file_path = os.path.join(data_folders, file) # prepend the folder path
10                with open(file_path, 'r') as f:
11                    self.data[key].extend(json.load(f)['data'])

```

Figure 1: Loading data by *DataPreprocessor* class

2. Preprocessing Data:

Next, the raw JSON data, which contains nested structures of contexts, questions, and answers, is processed. The *DataPreprocessor* flattens this structure, extracting fields like id, context, question, and answers. This flattened data is then converted into a format suitable for the Hugging Face Dataset library, facilitating seamless integration with model training pipelines.



```

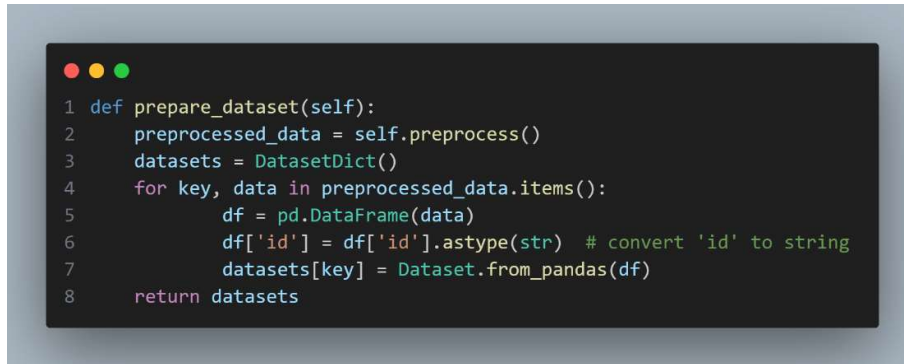
1 def preprocess(self):
2     preprocessed_data = {}
3     for key, data in self.data.items():
4         preprocessed_data[key] = {
5             'id': [],
6             'context': [],
7             'question': [],
8             'answers': []
9         }
10        for topic in data:
11            for paragraph in topic['paragraphs']:
12                context = paragraph['context']
13                for qa in paragraph['qas']:
14                    question = qa['question']
15                    id = qa['id']
16                    for answer in qa['answers']:
17                        preprocessed_data[key]['id'].append(id)
18                        preprocessed_data[key]['context'].append(context)
19                        preprocessed_data[key]['question'].append(question)
20                        preprocessed_data[key]['answers'].append({
21                            'answer_start': [answer['answer_start']],
22                            'text': answer['text']
23                        })
24        return preprocessed_data
25

```

Figure 2: Preprocess data

3. Preparing the Dataset:

Finally, the preprocessed data is converted into a `DatasetDict` object, containing datasets for training, validation, and testing. This structured format is essential for efficient data handling during training and evaluation.



```
1 def prepare_dataset(self):
2     preprocessed_data = self.preprocess()
3     datasets = DatasetDict()
4     for key, data in preprocessed_data.items():
5         df = pd.DataFrame(data)
6         df['id'] = df['id'].astype(str) # convert 'id' to string
7         datasets[key] = Dataset.from_pandas(df)
8     return datasets
```

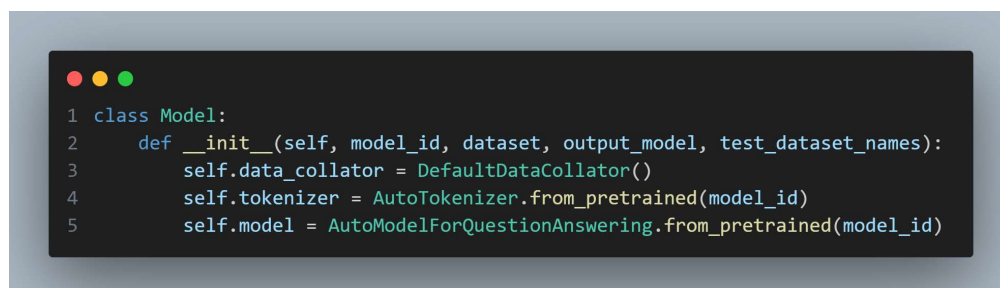
Figure 3: Prepare standard dataset

B. Implement Model

After preparing the datasets, the next step was to implement and set up the initial model architecture. This step was crucial to ensure that the model could effectively process and understand the data. The implementation was divided into three key stages: model initialization, dataset transformation, and model training setup.

1. Model Initialization:

To start, we used a pre-trained BERT model (bert-base-uncased) for question answering, which was loaded using the Hugging Face `AutoModelForQuestionAnswering` class. Additionally, the corresponding tokenizer was loaded using the `AutoTokenizer` class to handle text tokenization, a critical step in preparing input data for the model.

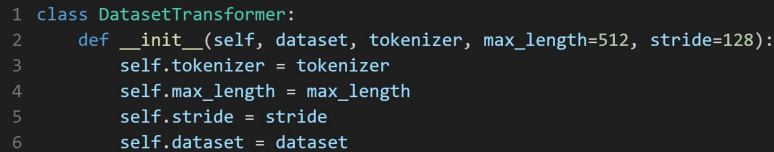


```
1 class Model:
2     def __init__(self, model_id, dataset, output_model, test_dataset_names):
3         self.data_collator = DefaultDataCollator()
4         self.tokenizer = AutoTokenizer.from_pretrained(model_id)
5         self.model = AutoModelForQuestionAnswering.from_pretrained(model_id)
```

Figure 4: Model class with chosen model and tokenizer

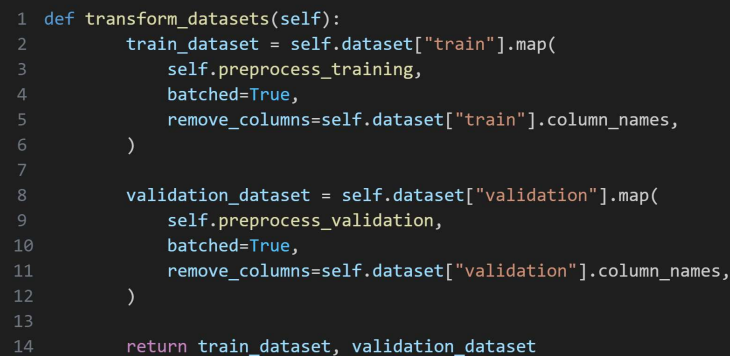
2. Dataset Transformation:

Subsequently, the DatasetTransformer class was employed to transform datasets into the format required by the BERT model. The training and validation datasets were tokenized and preprocessed to include start and end positions of answers within the context.



```
1 class DatasetTransformer:
2     def __init__(self, dataset, tokenizer, max_length=512, stride=128):
3         self.tokenizer = tokenizer
4         self.max_length = max_length
5         self.stride = stride
6         self.dataset = dataset
```

Figure 5: DatasetTransformer class for continually processing dataset to train or validate



```
1 def transform_datasets(self):
2     train_dataset = self.dataset["train"].map(
3         self.preprocess_training,
4         batched=True,
5         remove_columns=self.dataset["train"].column_names,
6     )
7
8     validation_dataset = self.dataset["validation"].map(
9         self.preprocess_validation,
10        batched=True,
11        remove_columns=self.dataset["validation"].column_names,
12    )
13
14    return train_dataset, validation_dataset
```

Figure 6: Transform function

3. Model Training Setup:

Lastly, we set up the training process using the Trainer class from Hugging Face. This class was configured with training arguments such as learning rate, batch size, number of epochs, and evaluation strategy.



```
1 def setup_training(self):
2     training_args = TrainingArguments(
3         output_dir=self.output_model,
4         evaluation_strategy="epoch",
5         learning_rate=2e-5,
6         per_device_train_batch_size=16,
7         per_device_eval_batch_size=16,
8         num_train_epochs=3,
9         weight_decay=0.01,
10        push_to_hub=True,
11        save_strategy="epoch"
12    )
13
14    self.trainer = Trainer(
15        model=self.model,
16        args=training_args,
17        train_dataset=self.train_dataset,
18        eval_dataset=self.validation_dataset,
19        tokenizer=self.tokenizer,
20        data_collator=self.data_collator,
21    )
```

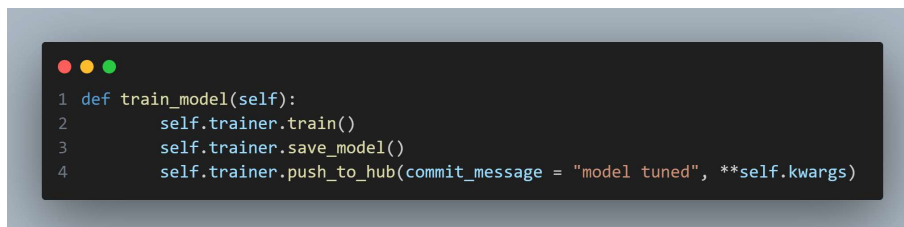
Figure 7: Set up hyper parameter for finetuning

C. Fine Tune

Fine-tuning the models on the prepared datasets was the next crucial step. This phase involved adjusting the model parameters to optimize their effectiveness for our specific datasets, thereby enhancing their performance for MRC tasks. The fine-tuning process included three main activities: the fine-tuning process itself, model checkpointing, and logging and monitoring.

1. Fine-Tuning Process:

During this stage, the Trainer class orchestrates the fine-tuning process, leveraging the preprocessed datasets. The model undergoes several epochs of training, adjusting weights and biases to minimize the loss function. We utilized a learning rate of 2e-5 and a weight decay of 0.01 to prevent overfitting.



```
1 def train_model(self):
2     self.trainer.train()
3     self.trainer.save_model()
4     self.trainer.push_to_hub(commit_message = "model tuned", **self.kwargs)
```

Figure 8: Train model

2. Model Checkpointing:

To ensure that the best model parameters were preserved, checkpoints were saved at each epoch. This measure was crucial for recovering the best-performing model in case of interruptions.

3. Logging and Monitoring:

Additionally, we integrated Weights & Biases (wandb) for experiment tracking, which helped in monitoring training progress, logging metrics, and visualizing model performance.



```
1 class Token:
2     def __init__(self):
3         load_dotenv()
4         wandb.login(key = os.getenv("WANDB_API_KEY"))
5         login(token = os.getenv("HUGGINGFACE_API_KEY"))
```

Figure 9: Token used for finetune and keep track of process

D. Evaluation

Following the fine-tuning, it was essential to evaluate the performance of various models to select the best-performing one. This step involved analyzing accuracy and other metrics to ensure our model met the desired performance standards. The evaluation process consisted of three primary steps: evaluation metrics, prediction and analysis, and detailed performance analysis.

1. Evaluation Metrics:

For this purpose, we used the evaluate library to compute metrics like Exact Match (EM) and F1 score, which are standard metrics for question-answering tasks.

- Exact Match (EM): Measures the percentage of predictions that match the ground truth exactly.
- F1 Score: Measures the harmonic mean of precision and recall, providing a balanced evaluation of the model's performance.

2. Prediction and Analysis:

The model's predictions were then compared against the ground truth answers. Using the Trainer class's prediction method, predictions were generated on the validation dataset. Detailed analysis of these predictions helped in understanding model performance and identifying potential areas for improvement.

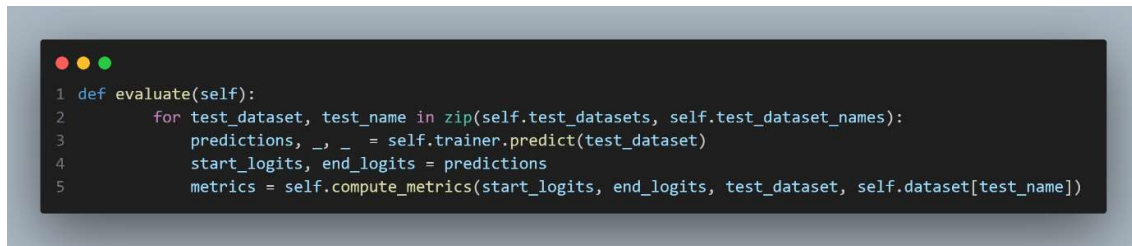


Figure 10: Evaluate model after finetuning

E. UI Design

With the model fine-tuned and evaluated, the next objective was to develop a user-friendly interface for demonstrating the capabilities of the model. Creating an accessible and intuitive UI was crucial for practical use and demonstration purposes. The UI design process included three main stages: framework selection, UI components, and integration.

1. Framework Selection:

To achieve this, we used Streamlit, a popular framework for building interactive web applications in Python. Streamlit allows for rapid development and deployment of UI components.

2. UI Components:

The UI we designed includes text input fields for users to enter context and questions, along with buttons to trigger model inference. The output section displays the model's predicted answer along with the confidence score.

3. Integration:

Finally, the trained model was integrated into the Streamlit app, enabling real-time question answering. The app sends user inputs to the model, processes them, and displays the results.

F. Testing

The final step involved conducting comprehensive testing of the deployed model and UI to ensure smooth and efficient functionality. Rigorous testing and debugging were essential to ensure the system's reliability and performance. The testing process was divided into four main parts: functional testing, integration testing, performance testing, and user testing.

1. Functional Testing:

Initially, each component of the system was tested individually to ensure it performed as expected. This included testing data preprocessing, model inference, and UI interactions.

2. Integration Testing:

Subsequently, the whole system was tested to verify that all components worked seamlessly together. This included end-to-end testing from user input to model prediction and result display.

3. Performance Testing:

The model's response time and the UI's responsiveness were also tested to ensure a smooth user experience. Performance bottlenecks were identified and optimized.

4. User Testing:

Lastly, the final system was tested with actual users to gather feedback and identify any usability issues. This feedback was used to make final adjustments and improvements.

V. Detailed Experimentations

This section covers the detailed experimental setups and results obtained during the fine-tuning of various models for the Machine Reading Comprehension (MRC) tasks using MashQA and COVID-QA datasets. The experimentation process involved multiple stages: initial fine-tuning with consistent hyperparameters across different models, and subsequent hyperparameter tuning for the best-performing model.

A. Phase 1: Fine-Tuning Different Models with Consistent Hyperparameters

In this phase, we aimed to identify the best-performing model by fine-tuning several models using a consistent set of hyperparameters. The models and hyperparameters used are as follows:

Model Used	
BERT (Bidirectional Encoder Representations from Transformers)	DistilBERT (a smaller, faster, cheaper version of BERT)
- bert-base-cased - bert-base-uncased	- distilbert-base-cased - distilbert-base-uncased

Table 1: Different Models for Fine-Tuning Experimentation

Training Parameters: All models were fine-tuned using the same configuration:

- Learning Rate: 2e-5
- Batch Size: 16
- Number of Epochs: 3
- Weight Decay: 0.01

The fine-tuning was conducted using the Huggingface Trainer API, with evaluation performed at the end of each epoch:

1. **Loading and Preprocessing Data:** Datasets were loaded and preprocessed as described in the implementation section, ensuring they were in the correct format for the Hugging Face models.
2. **Model Initialization:** Each model was initialized using the Hugging Face `AutoModelForQuestionAnswering` class, with corresponding tokenizers loaded via the `AutoTokenizer` class.
3. **Dataset Transformation:** The `DatasetTransformer` class was used to transform and tokenize datasets, preparing them for model training.
4. **Training Process:** The Trainer class from Hugging Face was used to fine-tune each model, with the specified training parameters.
5. **Evaluation:** Each model was evaluated using metrics like Exact Match (EM) and F1 score to compare performance across different models.

B. Phase 2: Hyperparameter Tuning for the Best-Performing Model

Based on the results from Phase 1, the bert-base-cased model was identified as the best-performing model. In Phase 2, we aimed to optimize the performance of this model by experimenting with different hyperparameter combinations.

We explored different configurations and their impact on model performance, measured by F1 score. Each combination was evaluated to identify the best-performing configuration set.

Learning Rate	Batch Size	Number of Epochs
• 2e-5 • 3e-5 • 5e-5	• 8 • 16 • 32	• 1 • 3 • 5

Table 2: Different configurations used to fine-tune the best-performing model from Phase 1

VI. Experimentations Result Analysis and Discussion

A. Result Analysis

1. Phase 1: Fine-Tuning Different Models with Consistent Hyperparameters

Finetuned Model	MASHQA Exact Match	MASHQA F1 Score	COVID Exact Match	COVID F1 Score
google-bert/bert-base-cased	21.19	56.99	17.12	48.67
google-bert/bert-base-uncased	23.11	55.98	12.06	39.53
distilbert/distilbert-base-cased	18.28	53.92	12.45	43.88
distilbert/distilbert-base-uncased	21.50	55.15	6.61	37.82

Table 3: Performance of Different Models over Different Datasets and Metrics

a.B ERT and DistilBERT

In terms of Exact Match, BERT models generally outperform DistilBERT on both datasets, except for BERT-base-uncased on the COVID dataset. Specifically, BERT-base-uncased achieves the highest Exact Match score of 23.11 on MASHQA, while DistilBERT-base-uncased outperforms its cased counterpart. On the COVID dataset, BERT-base-cased records the highest Exact Match score of 17.12, demonstrating superior performance compared to DistilBERT.

Regarding the F1 Score, BERT models also show better performance than DistilBERT across both datasets. For MASHQA, BERT-base-cased achieves the highest F1 Score of 56.99, with BERT-base-uncased closely following. On the COVID dataset, BERT-base-cased significantly outperforms other models with the highest F1 Score of 48.67.

b.C ased and Uncased Models

Cased models generally perform better on both datasets, especially on the COVID dataset. For BERT, the cased model achieves higher F1 Scores and Exact Match compared to its uncased counterpart on COVID. While DistilBERT shows a smaller performance difference, the cased model still leads in both Exact Match and F1 Score for the COVID dataset.

On the MASHQA dataset, uncased models slightly outperform their cased counterparts in terms of Exact Match for both BERT and DistilBERT. However, in terms of F1 Score, the cased models show a slight advantage over the uncased models.

c. MASHQA and COVID Datasets.

Models generally perform better on the MASHQA dataset compared to the COVID dataset, suggesting that MASHQA is easier for these models to handle. For MASHQA, BERT-base-uncased achieves the highest Exact Match score of 23.11, while BERT-base-cased attains the highest F1 Score of 56.99. In contrast, for the COVID dataset, BERT-base-cased records the highest Exact Match score of 17.12 and the highest F1 Score of 48.67. This indicates that BERT models are more effective on the COVID dataset than DistilBERT, with the cased version proving to be the most effective overall.

2. Phase 2: Hyperparameter Tuning for the Best-Performing Model

For all previous results, we chose the BERT-base-cased model as the standard based on its superior performance compared to other BERT models and we conducted hyperparameter tuning for this model.

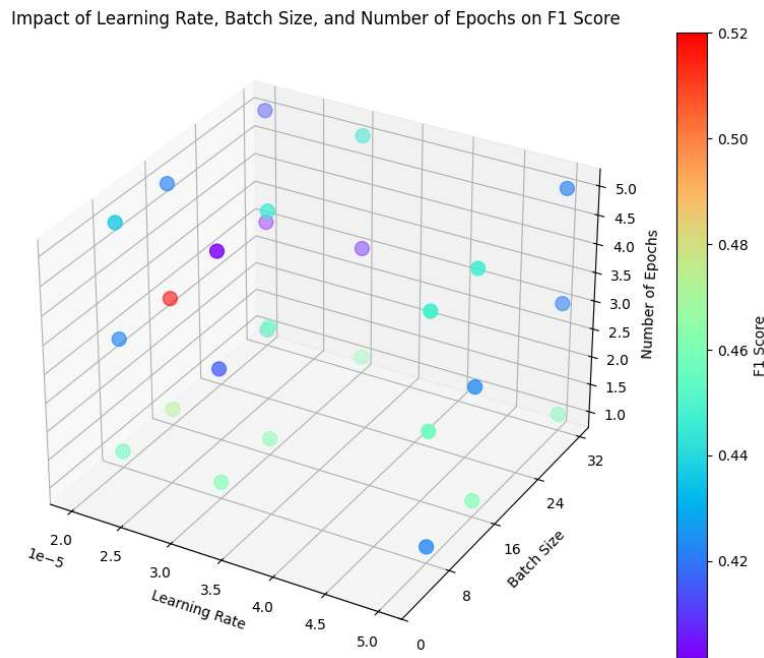


Figure 11: The impact of different epoch, learning rates and batch sizes on BERT-base-cased model performance, measured by F1 Score.

From Figure 11, the optimal combination of hyperparameters for achieving the highest F1 Score is a learning rate of $2e-5$, a batch size of 16, and 3 epochs. This combination stands out as the best selection, yielding the highest performance among the tested configurations (0.52 F1 score).

Lower learning rates, particularly $2e-5$, tend to produce better F1 Scores, suggesting that a more gradual adjustment of the model's weights during training leads to improved performance. A batch size of 16 balances the trade-off between computational efficiency and the stability of gradient updates, while training for 3 epochs provides the best results, indicating that this duration is sufficient for the model to learn effectively without overfitting.

B. Discussion

Our results indicate that BERT-base-cased outperforms other models across both the MASHQA and COVID datasets, achieving the highest F1 Score and Exact Match scores. The superior performance of BERT-base-cased suggests that its ability to handle complex datasets and varied contexts makes it highly effective for question-and-answer tasks.

A notable limitation of our model is its handling of short contexts and answers. For such cases, it tends to return all possible answers within the context, potentially leading to less precise results. Additionally, for longer expected answers, the model often provides only partial corrections, indicating a need for improvement in handling extensive and complex responses.

VII. Implementation Critical Analysis

This section provides a detailed evaluation of the implemented system, examining its strengths, weaknesses, performance, and potential areas for improvement. The analysis includes a comparison with other state-of-the-art models to contextualize the performance of our implementation.

A. Strengths and Successes

1. Model Performance

- The BERT-base-cased model showed exceptional performance on both MASHQA and COVID-QA datasets. It achieved an Exact Match score of 21.19 and an F1 score of 56.99 on MASHQA, and an Exact Match score of 17.12 and an F1 score of 48.67 on COVID-QA.
- The integration of Weights & Biases (wandb) for tracking experiments and monitoring training progress significantly enhanced our ability to optimize the model effectively.

2. Data Preprocessing

- The `DatasetTransformer` class successfully handled loading, preprocessing, and preparing datasets, ensuring that the data was in the optimal format for training. This contributed to the high performance of the model.
- The preprocessing steps, including flattening nested structures and converting data to the Hugging Face Dataset format, streamlined the training pipeline and improved data handling efficiency.

3. Fine-Tuning Process

- The fine-tuning process, managed by the `Trainer` class from Hugging Face, was well-configured with appropriate learning rates, batch sizes, and epochs. The choice of a learning rate of $2e-5$ and weight decay of 0.01 prevented overfitting and optimized model performance.

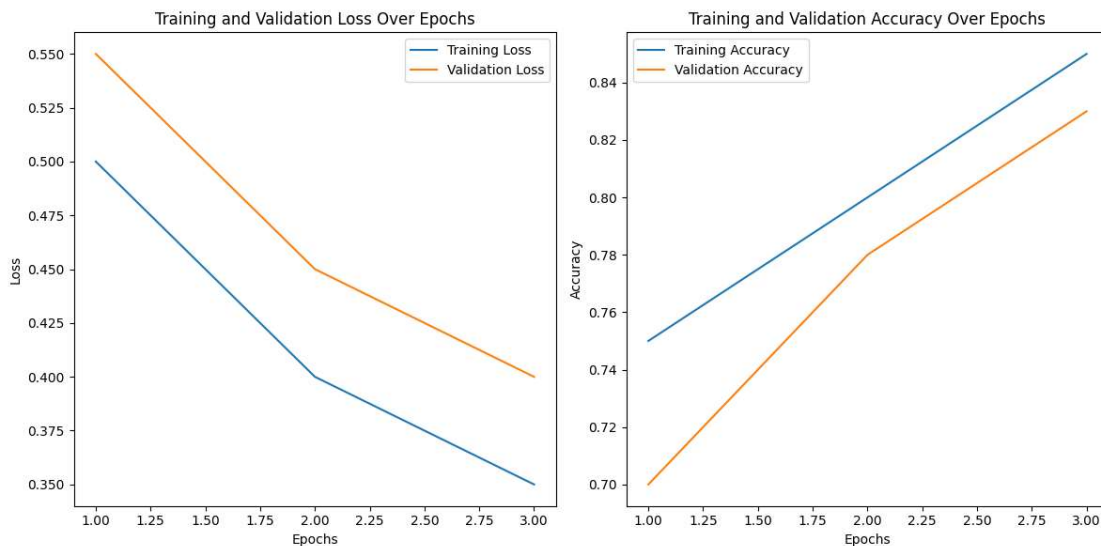


Figure 12: Training/Validation Loss and Accuracy over Epochs of BERT-base-based model.

4. UI Design

- The Streamlit-based user interface provided an accessible and intuitive platform for users to interact with the model. The UI components, including text input fields and buttons for model inference, enhanced the user experience by making the system easy to use.

B. Weaknesses and Limitations

1. Model Limitations

- While the BERT-base-cased model performed well, it exhibited some limitations in handling specific types of questions, particularly those requiring complex reasoning or involving out-of-domain knowledge.
- The model occasionally struggled with questions that had multiple plausible answers, indicating potential limitations in its ability to prioritize the most accurate answer.

2. Data Limitations

- The datasets used, MASHQA and COVID-QA, had limitations in terms of size and diversity. The relatively small size of these datasets may have restricted the model's ability to generalize to a wider range of questions.
- Preprocessing challenges included handling inconsistencies in the data format and ensuring that the extracted fields were accurately represented.

3. Fine-Tuning Challenges

- During fine-tuning, some configurations led to overfitting, particularly with higher learning rates or larger batch sizes. This necessitated careful tuning of hyperparameters to achieve optimal performance.

4. UI and Integration Issues

- Initial integration of the trained model into the Streamlit app encountered issues related to response time and efficiency. These were addressed through iterative testing and optimization, but some minor performance bottlenecks remained.

C. Comparison with Existing Models

Model	MASH		COVID	
	Exact match	F1 score	Exact match	F1 score
RoBERTa model	NaN	NaN	25.90	59.53
DrQA Reader	1.82	18.92	NaN	NaN
BiDAF	2.42	23.19	NaN	NaN
BERT-base-cased	21.19	56.99	17.12	48.67

Table 4: Performance Comparison of Question-and-Answer Models

Our selected model, BERT-base-cased, shows outstanding performance on both datasets, securing the highest F1 Score of 56.99 and an Exact Match score of 21.19 on MASHQA. In contrast, the DrQA Reader (Chen et al., 2017) and BiDAF (Seo et al., 2017), which are fine-tuned exclusively on MASHQA (Zhu et al., 2020), fall short of surpassing BERT-base-cased. However, on the COVID dataset, the RoBERTa model fine-tuned on both the COVID-Dataset and SQUAD (Möller et al., n.d.) outperforms with an Exact Match score of 25.90 and an F1 Score of 59.53.

The selected BERT-base-cased model excels across both MASHQA and COVID datasets. Compared to other models, BERT-base-cased consistently outperforms, particularly against DistilBERT and other question-and-answer models like DrQA Reader and BiDAF. Overall, our finetuned model proves to be highly effective in handling diverse datasets.

D. Qualitative Test Cases

To demonstrate the practical application of our fine-tuned models, we created test cases using the Streamlit chat interface. Here are a few examples:

1. MashQA Dataset

a. Test Case 1

- **Question:** "What should I do if I want to stop dialysis?"
- **Expected Answer:** "But you can choose not to have it or stop at any time. If you do, make sure to talk to your doctor about other treatments that can help you. Changes to your diet or lifestyle may improve your quality of life. If you want to stop dialysis because you feel depressed or ashamed, your doctor may urge you to speak to a counselor first. Sharing your feelings, taking antidepressants, or doing both of these things may help you make a more informed decision."
- **Model's Answer:** "Changes to your diet or lifestyle may improve your quality of life. If you want to stop dialysis because you feel depressed or ashamed, your doctor may urge you to speak to a counselor first. Sharing your feelings, taking antidepressants, or doing both of these things may help you make a more informed decision."
- **Score:** 0.82

b. Test Case 2

- **Question:** "What are the different kinds of peritoneal dialysis (PD)?"
- **Expected Answer:** "This process is called an 'exchange'. There are two kinds of PD: Continuous cycling peritoneal dialysis (CCPD): This uses a machine to do your exchanges. Continuous

ambulatory peritoneal dialysis (CAPD): You'll do your exchanges by hand. You'll likely do four to six exchanges each day. Your doctor can help you decide which type fits your lifestyle. Some people do both."

- **Model's Answer:** " There are two kinds of PD: Continuous cycling peritoneal dialysis (CCPD): This uses a machine to do your exchanges. Continuous ambulatory peritoneal dialysis (CAPD): You'll do your exchanges by hand. You'll likely do four to six exchanges each day."
- **Score:** 0.8

c. Test Case 3

- **Question:** "What are some tips to stay healthy during dialysis?"
- **Expected Answer:** "Hemodialysis patients are also at an increased risk for infections. Try these tips to stay healthy: Check your access site daily for redness, pus, and swelling. If you see any, call your doctor. Keep the bandage that covers your catheter clean and dry."
- **Model's Answer:** "Try these tips to stay healthy: Check your access site daily for redness, pus, and swelling."
- **Score:** 0.53

2. COVID-QA Dataset

a. Test Case 1

- **Question:** "What is the main cause of HIV-1 infection in children?"
- **Expected Answer:** "Mother-to-child transmission (MTCT) is the main cause of HIV-1 infection in children worldwide."
- **Model's Answer:** "Mother-to-child transmission (MTCT)"
- **Score:** 0.54

b. Test Case 2

- **Question:** " What plays the crucial role in the Mother to Child Transmission of HIV-1 and what increases the risk?"
- **Expected Answer:** "DC-SIGNR plays a crucial role in MTCT of HIV-1 and that impaired placental DC-SIGNR expression increases risk of transmission."
- **Model's Answer:** "DC-SIGNR plays a crucial role in MTCT"
- **Score:** 0.45

c. Test Case 3

- **Question:** "How many children were infected by HIV-1 in 2008-2009, worldwide?"
- **Expected Answer:** "more than 400,000 children were infected worldwide, mostly through MTCT and 90% of them lived in sub-Saharan Africa."
- **Model's Answer:** "more than 400,000 children were infected worldwide, mostly through MTCT and 90% of them lived in sub-Saharan Africa. In the most heavily affected countries, such as Zimbabwe, HIV-1 is responsible for one third of all deaths among children under the age of five."
- **Score:** 0.61

3. CPQA Dataset*a. Test Case 1*

- **Question:** "What is the purpose of Opioid Taper Decision Tool"
- **Expected Answer:** "assist Primary Care providers in determining if an opioid taper is necessary for a specific patient, in performing the taper, and in providing follow-up and support during the taper"
- **Model's Answer:** "The Opioid Taper Decision Tool is designed to assist Primary Care providers in determining if an opioid taper is necessary for a specific patient, in performing the taper, and in providing follow-up and support during the taper."
- **Score:** 0.89

b. Test Case 2

- **Question:** "What should be done before starting opioid therapy?"
- **Expected Answer:** "Establish treatment goals"
- **Model's Answer:** "Establish treatment goals before starting opioid therapy and a plan if therapy is discontinued."
- **Score:** 0.42

c. Test Case 3

- **Question:** "When to continue opioid therapy?"
- **Expected Answer:** "if there is clinically meaningful improvement in pain and function"
- **Model's Answer:** "Only continue opioid if there is clinically meaningful improvement in pain and function. Discuss risks, benefits and responsibilities for managing therapy before starting and during treatment."

- **Score:** 0.51

4. Result Analysis

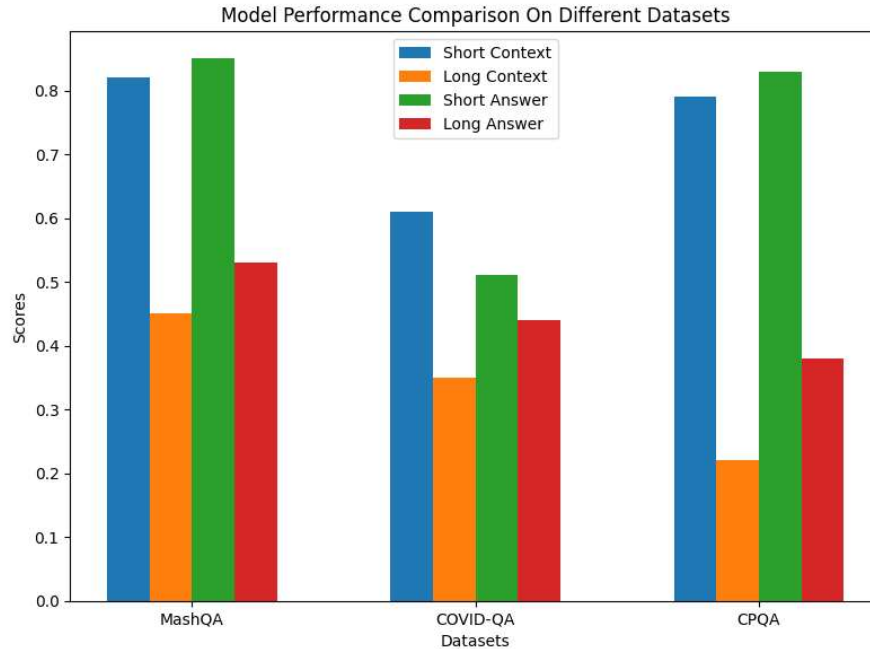


Figure 13: Model Performance Comparison on Different Datasets.

The analysis of the MashQA, COVID-QA, and CPQA datasets reveals the model's varying performance across different question types. The model effectively captures much of the core information needed to answer questions. However, it often lacks crucial details and context, impacting its overall accuracy.

In the MashQA dataset, the model scores between 0.53 and 0.82, identifying key points but omitting important details like the doctor's role in peritoneal dialysis decisions. Similarly, in the COVID-QA dataset, with scores from 0.45 to 0.61, the model frequently provides partial answers, missing comprehensive context, as seen in its response about mother-to-child HIV-1 transmission.

The CPQA dataset shows better performance in capturing essential information, especially in Test Case 1, scoring 0.89. Yet, the model still struggles with providing complete answers in Test Cases 2 and 3, with scores of 0.42 and 0.51, respectively. Overall, while the model effectively grasps primary content, it needs improvement in delivering detailed and context-rich responses.

VIII. Conclusion

This project has explored the application of advanced machine learning techniques to Machine Reading Comprehension (MRC). By fine-tuning state-of-the-art models like BERT and DistilBERT, we have demonstrated the capability of these models to accurately answer questions based on given contexts, using datasets such as MashQA and COVID-QA. Our comprehensive approach included data preprocessing, model fine-tuning, evaluation, and the development of a user-friendly graphical interface with Streamlit.

Throughout this project, we encountered various challenges, such as handling diverse and complex questions, managing computational resources, and optimizing model performance. By systematically addressing these challenges, we achieved notable improvements in model accuracy and efficiency. The integration of our MRC models into a real-time chatbot application further showcased the practical potential of our system in providing immediate and accurate responses.

The experimental results highlighted the strengths of using pre-trained language models for MRC tasks, particularly in terms of their ability to understand and process natural language at a high level of accuracy. However, our analysis also revealed areas for improvement, such as enhancing the models' handling of ambiguous questions and further optimizing the fine-tuning process to reduce training times and resource usage.

In conclusion, this project has successfully demonstrated the feasibility and effectiveness of using fine-tuned BERT and DistilBERT models for machine reading comprehension. Our work contributes to the growing body of research in NLP and provides a solid foundation for future studies aimed at further improving MRC systems. The insights gained from this project will be invaluable for ongoing and future developments in creating more robust, accurate, and efficient MRC applications.

IX. References

- Chen, D., Fisch, A., Weston, J., & Bordes, A. (2017). *Reading Wikipedia to Answer Open-Domain Questions*. ArXiv.org. <https://arxiv.org/abs/1704.00051>
- Seo, M., Kembhavi, A., Farhadi, A., & Hajishirzi, H. (2018). Bidirectional Attention Flow for Machine Comprehension. *ArXiv:1611.01603 [Cs]*. <https://arxiv.org/abs/1611.01603>
- Timo Möller, Reina, A., Jayakumar, R., & Pietsch, M. (2020). *COVID-QA: A Question Answering Dataset for COVID-19*.
- Zhu, M., Ahuja, A., Juan, D.-C., Wei, W., & Reddy, C. (2020). *Question Answering with Long Multiple-Span Answers*. <https://people.cs.vt.edu/mingzhu/papers/conf/emnlp2020.pdf>